

Resumen Completo: Parte 1 - Texture Painting en 2D

El objetivo de la Parte 1 del TP fue construir un sistema de pintura bidimensional (similar a MS Paint) interactuando directamente sobre la imagen que luego se utiliza como textura para el modelo 3D. A continuación se detallan todos los problemas planteados, la justificación matemática y el código exacto implementado para cada solución.

1. El Lienzo y el Mapeo de Coordenadas

Cuando el usuario hace click en la ventana secundaria (la que muestra la textura 2D plana), el sistema operativo y la librería GLFW informan la posición del puntero (`xpos` , `ypos`) en base a píxeles de pantalla, donde el eje `y=0` se encuentra en el **borde superior**.

Sin embargo, la imagen en memoria RAM donde realmente pintamos los colores, almacena sus datos en formato de filas y columnas, y en el mundo de los gráficos por computadora el eje `y=0` suele ubicarse en el **borde inferior**.

Solución en `auxMouseButtonCallback` : Para normalizar estas coordenadas, dividimos la posición del mouse por el tamaño total de la ventana y lo multiplicamos por el tamaño de la imagen real. Para el eje Y, invertimos el valor restándolo de la altura de la ventana.

```
cpp
// 1. Obtenemos el tamaño de la ventana
BufferSize bs = getBufferSize(window);

// 2. Normalizamos y pasamos a la escala de la imagen
float j = (xpos / bs.width) * image.GetWidth();
float i = ((bs.height - 1 - ypos) / bs.height) * image.GetHeight();
```

2. Rasterización de Segmentos Continuos: Algoritmo DDA

Al arrastrar el mouse muy rápido, los eventos que dispara GLFW generan puntos separados por huecos vacíos. Para rellenar esos huecos creando un trazo ininterrumpido, implementamos el **Algoritmo DDA** (Analizador Diferencial Digital).

La matemática del DDA consiste en:

1. Analizar las distancias Δx y Δy . El eje más largo se denomina "Eje Dominante".
2. Avanzar de a un (1) téxel entero a lo largo del eje dominante, garantizando que el trazo no deje ningún hueco.
3. Avanzar una fracción proporcional (Pendiente m) en el otro eje, redondeando el resultado `std::round()` para encender el téxel más cercano a la recta ideal matemática.
4. Prevenir trazos inversos: si el movimiento fue de derecha a izquierda o de arriba hacia abajo, intercambiamos los puntos iniciales y finales para garantizar un bucle que siempre avance con incrementos positivos.

Código de `drawLineDDA` :

cpp

```
void drawLineDDA(float x0, float y0, float x1, float y1){

    // Distancias recorridas en cada eje
    float dx = x1 - x0;
    float dy = y1 - y0;

    // Si el trazo es más horizontal (X domina)
    if (std::abs(dx) > std::abs(dy)){

        // Invertimos si dibujamos de derecha a izquierda
        if (dx < 0){
            std::swap(x0, x1);
            std::swap(y0, y1);
            dx = x1 - x0;
            dy = y1 - y0;
        }

        float y = y0;
        int x = std::round(x0);
        int x_final = std::round(x1);
        float m = dy / dx; // Pendiente

        for (; x <= x_final; ++x){
            // Estampamos el pincel
            drawCircle(x, y, radius);
            y += m; // Avanzamos fraccionariamente en Y
        }

    } else {

        // Si el trazo es más vertical (Y domina)

        // Invertimos si dibujamos de arriba hacia abajo
        if (dy < 0){
            std::swap(y1, y0);
            std::swap(x1, x0);
            dy = y1 - y0;
            dx = x1 - x0;
        }

        float x = x0;
        float inv_m = (dy != 0) ? (dx / dy) : 0.0f; // Pendiente invertida
        int y = std::round(y0);
        int y_final = std::round(y1);
```

```

        for (; y <= y_final; ++y){
            // Estampamos el pincel
            drawCircle(x, y, radius);
            x += inv_m; // Avanzamos fraccionalmente en X
        }
    }
}

```

3. Grosor del Pincel y Efecto Acuarela (Alpha Blending)

Para no pintar simples líneas de 1 píxel de espesor y permitir opacidad translúcida, abstraímos el concepto de "pintar un píxel" hacia una nueva función fundamental: `drawCircle`.

El problema de repintar téxeles y la acumulación de Alpha

Cuando el DDA avanza milimétricamente, estampar repetidos círculos genera que un mismo téxel reciba la operación de "Alpha Blending" de forma múltiple (superposición de pasadas). Con una opacidad de 0.50.5, en tan solo cuatro pasadas casi invisibles, el color sumaría un 94%94% de opacidad, destruyendo la transparencia.

La solución: Téxeles Visitados y Matriz 2D

Implementamos un rastreador (máscara booleana) que recuerda si el téxel ya recibió pintura durante el trazo de arrastre actual, limitando la aplicación de Alpha Blending a una única vez.

1. Variables Globales:

```

cpp
float radius = .5;
glm::vec4 color = { 0.f, 0.f, 0.f, 1.f };
bool show_back_buffer = false;

// Matriz 2D para recordar los téxeles pintados
std::vector<std::vector<bool>> visited_pixels;

```

2. Reiniciar el estado en el Callback (El Click Inicial): Al hacer click, generamos una matriz completamente limpia y llena de `false` con las dimensiones de la textura.

```
cpp
// En auxMouseButtonCallback (Y luego repetido en mainMouseButtonCallback)
if (action == GLFW_PRESS) {
    visited_pixels.assign(image.GetHeight(), std::vector<bool>(image.GetWidth(), false));
    // ...
}
```

La Función Definitiva: `drawCircle`

El círculo no se evalúa recorriendo toda la imagen. Generamos una "Caja Delimitadora" (*Bounding Box*) usando `std::floor` y `std::ceil` para asegurar que el escaneo cubra hasta el último borde fraccionario del radio. Luego validamos la ecuación del círculo $dx^2 + dy^2 \leq r^2$ $dx^2 + dy^2 \leq r^2$.

```

cpp
void drawCircle(float center_x, float center_y, float r) {
    // 1. Calcular caja delimitadora y redondear
    int x_min = std::floor(center_x - r);
    int x_max = std::ceil(center_x + r);
    int y_min = std::floor(center_y - r);
    int y_max = std::ceil(center_y + r);

    // 2. Escanear la cuadrícula
    for (int y = y_min; y <= y_max; ++y) {
        for (int x = x_min; x <= x_max; ++x) {

            float dx = x - center_x;
            float dy = y - center_y;

            // 3. Ecuación de distancia radial (Teorema de Pitágoras)
            if (dx * dx + dy * dy <= r * r) {

                int j = x; // columna
                int i = y; // fila

                // 4. Proteger bordes contra crasheos
                if (j >= 0 && j < image.GetWidth() && i >= 0 && i < image.GetHeight()) {

                    // 5. Máscara de Tóxeles Visitados
                    if (!visited_pixels.empty() && !visited_pixels[i][j]) {

                        visited_pixels[i][j] = true; // Anotar

                        // 6. Alpha Blending (Interpolación lineal / LERP)
                        glm::vec3 c_fondo = image.GetRGB(i, j);
                        glm::vec3 c_pincel = glm::vec3(color);
                        float alpha = color.a;

                        glm::vec3 c_final = (c_pincel * alpha) + (c_fondo * (1.0f -
alpha));

                        // 7. Guardar en Memoria RAM
                        image.SetRGB(i, j, c_final);
                    }
                }
            }
        }
    }
}

```

